# Mood Lamp

Harshvardhan

## 1 Introduction

The project involved the design and construction of a Bluetooth and potentiometer controlled RGB mood lamp using a high-power SMD RGB LED as the primary light source and an Arduino Nano as the main controller. The initial idea behind the project was to create a compact mood lighting device capable of producing a wide range of colors while offering two modes of control: local manual control using potentiometers and remote wireless control using Bluetooth from a smartphone. Colours have an important impact on human mood and emotions, and different lighting colors can create different psychological effects such as relaxation, focus, or calmness. Because of this relationship between lighting colour and emotional ambience, the device is referred to as a mood lamp. The system is powered using a USB-C type receptacle that supplies a regulated 5 V DC input to the circuit.

## 2 System Architecture

The core of the system is an Arduino Nano development board, which processes user inputs and generates PWM signals to control the brightness of the red, green, and blue channels of the LED.

### 2.1 Light Source and Driver

The RGB LED used in the project is a 3 W high power SMD device in which the three internal LEDs (red, green, and blue) can be independently controlled to create different colors through additive color mixing.

The LED assembly was constructed on a small zero PCB that served as a dedicated mounting board for the RGB LED. A circular hole was carefully cut in the PCB with a diameter approximately equal to that of the RGB LED package. The SMD RGB LED was then positioned in this opening and soldered from the back side of the PCB so that the light-emitting surface remained unobstructed on the front side. This mounting arrangement ensured proper alignment of the LED while allowing the light to emerge cleanly through the opening.

Electrically, the three anode terminals of the RGB LED were connected together on the PCB so that the LED effectively operated as a common-anode type RGB LED, while the individual cathode terminals corresponding to the red, green, and blue channels were routed separately to the driver circuitry so that each color channel could be independently controlled.

The RGB LED is driven through a ULQ2003A driver IC, which is a Darlington transistor array used to provide sufficient current driving capability and to isolate the microcontroller from the high-power load.

## 2.2 Thermal Management

Because the LED is high power and dissipates significant heat, thermal management was necessary.

After soldering the LED terminals, an insulating layer of electrical tape was placed over the soldered connections. This insulation was necessary because an aluminium block was mounted directly behind the PCB to act as a heat sink. When the heat sink was later fixed using screws, the insulating tape ensured that the metallic surface of the heat sink could not accidentally come into contact with the exposed solder joints of the LED terminals, thereby preventing possible short circuits.

To further improve thermal performance, thermal paste was applied between the LED mounting PCB and the aluminium heat sink block. The heat sink assembly was then mechanically secured to the enclosure using screws and nuts to provide effective heat dissipation during operation. Despite using a heat sink and thermal paste, it was observed during testing that the LED became very hot when driven at full intensity, and sometimes one of the RGB channels would temporarily turn off due to overheating, but would resume functioning after the LED cooled down. To mitigate this problem, instead of selecting resistors that allowed the LED to operate at full brightness (255 PWM level), resistor values were chosen so that the LED effectively operated at approximately 150/255 of its maximum intensity, thereby reducing power dissipation and improving reliability. Under this operating condition, the overall maximum current requirement of the LED at full combined RGB intensity was measured to be approximately 550 mA.

# 3 Control Interfaces

## 3.1 Local Control

For local control of the lamp, three panel-mount potentiometers were installed on the enclosure, each corresponding to the red, green, and blue color channels. These potentiometers provide analog voltage signals that are read by the Arduino Nano through its analog input pins. Since potentiometers can introduce noise and small fluctuations in the analog readings, three 0.1 µF capacitors were connected to filter the signals and reduce noise.

In addition, software filtering was implemented using a rolling average algorithm with a window size of eight samples to smooth the potentiometer readings and produce stable brightness transitions. A deadzone threshold was also implemented in the software so that very small PWM values below a defined threshold are forced to zero to prevent faint flickering or unintended illumination when the potentiometer is near its minimum position. The analog values from the potentiometers are read by the Arduino's 10-bit ADC, which produces values between 0 and 1023. These values are converted to 8-bit PWM values between 0 and 255 by performing a bit shift operation (right shift by two bits). The PWM signals are then applied to the LED driver outputs connected to the Arduino pins controlling the red, green, and blue LED channels.

### 3.2 Wireless Control

For wireless control, an HC-05 Bluetooth module was integrated into the system. The module communicates with the Arduino using UART serial communication implemented through the SoftwareSerial library, using two digital pins of the Arduino as RX and TX lines. The Bluetooth module operates at a baud rate of 9600. To reduce the vertical height of the internal assembly and improve mechanical stability inside the enclosure, the HC-05 module was mounted horizontally on the zero PCB using a 90-degree female Berg strip connector. This orientation helped in keeping the layout compact and prevented mechanical stress on the module during enclosure assembly.

The lamp can be controlled wirelessly using a smartphone application called Serial Bluetooth Terminal, which communicates with the Arduino over the Bluetooth serial link. In Bluetooth mode, the application transmits RGB intensity values in the format R,G,B, where each value ranges from 0 to 255 corresponding to the PWM intensity levels of the red, green, and blue channels respectively. The application must be configured to send both carriage return and line feed characters (CR+LF) at the end of each transmission to ensure proper parsing of the received data. The Arduino reads these transmitted values using the parseInt() function and updates the LED brightness accordingly.

A toggle switch is used to select the mode of operation between local potentiometer control and Bluetooth control. When the mode switch indicates local mode, the Arduino reads the filtered potentiometer values and uses them for LED control, whereas in Bluetooth mode it ignores the potentiometers and uses the received RGB values from the Bluetooth module.

## 4 Special Features: Flickering Effect

Another toggle switch is used to enable or disable a flickering effect. When the flicker switch is turned off, the LED glows steadily according to the selected RGB intensities. When the flicker switch is turned on, the LED produces a flickering effect that intermittently turns the LED on and off.

This flickering effect is generated using a 64-bit Linear Feedback Shift Register (LFSR) implemented in software to produce pseudo-random numbers. The LFSR uses taps at bit positions corresponding to bits 0, 1, 3, and 4 of the register. At each iteration of the program loop, the feedback bit is calculated using XOR operations on these taps, and the register is shifted to generate the next pseudo-random value.

The least significant bit of the LFSR is used to decide whether the LED should remain on or off at that moment, producing a natural-looking random flicker effect. To ensure that the LFSR does not start with a deterministic value, a pseudo-random seed is generated during startup by repeatedly reading analog noise from an unused analog input pin and assembling these random bits into the initial 64-bit register value. If the seed accidentally becomes zero, a predefined non-zero value is used because an LFSR initialized to zero would remain stuck at zero. The LED flickering effect is produced by turning the LED on when the LFSR least significant bit is 1 and turning it off when the bit is 0, with a short delay of about 20 milliseconds between flicker states to control the flicker speed.

# 5 Mechanical Assembly

All electronic components including the Arduino Nano, HC-05 module, ULQ2003A driver IC, and the resistors were soldered onto a main zero PCB using hook-up wires for interconnections. To allow modular assembly and easier maintenance, female JST connectors were soldered onto the zero PCB so that external components such as potentiometers, switches, and the LED assembly could be connected or disconnected easily. Plastic insulated multi-strand wires were used for all connections between the PCB and external components to ensure flexibility and durability. Some groups of wires were bundled together using heat-shrink tubing to maintain neatness and improve cable management inside the enclosure.

The entire system was housed inside an IP65-rated plastic enclosure to provide mechanical protection and environmental resistance. The main zero PCB containing the control electronics was mounted at the base of the enclosure using screws and nuts. The LED assembly mounted on the small zero PCB and heat sink was fixed near the top of the enclosure. A large circular hole was cut on the top surface of the enclosure to allow the LED light to emerge. To diffuse the light and produce a soft uniform glow rather than a harsh point source, a table tennis ball was cut in half and used as a diffuser. This half table tennis ball was attached over the LED opening using two-part epoxy adhesive.

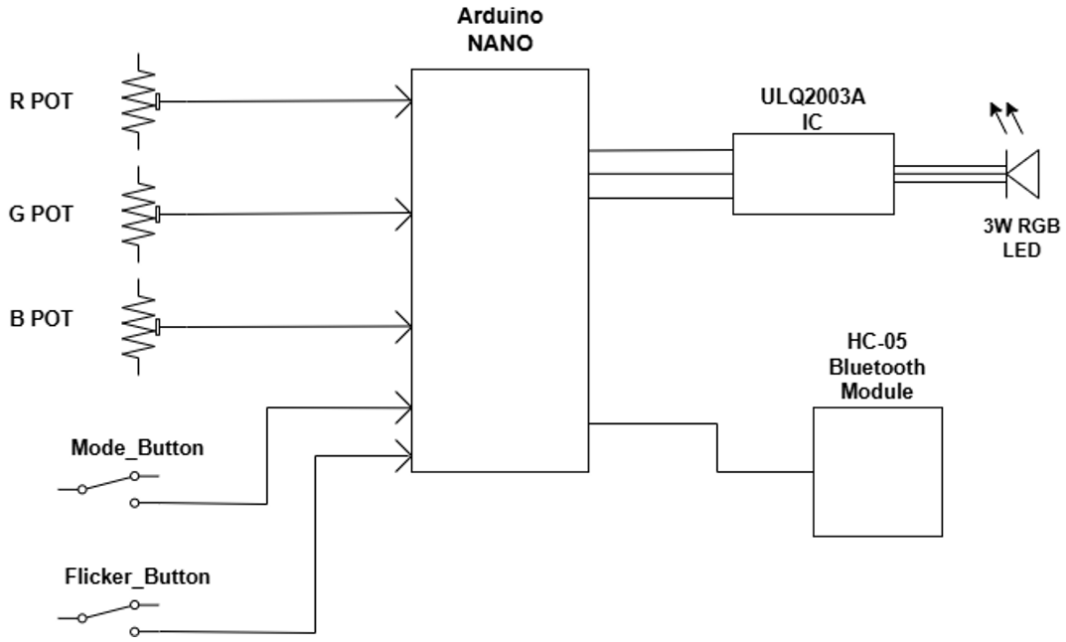
Additional holes were drilled in the enclosure using a drill machine with appropriately sized drill bits to mount the potentiometers, SPDT toggle switches, and the USB-C receptacle. The USB-C connector was secured in place using epoxy adhesive to ensure it remained firmly attached to the enclosure during repeated insertion and removal of power cables. For improved usability and professional appearance, custom 3D-printed decals were prepared and attached to the enclosure using Elfy adhesive to clearly label the controls and switches.

After completing the mechanical assembly and electrical connections, the firmware was uploaded to the Arduino Nano through the Arduino development environment. The completed device allows users to create various lighting moods by adjusting RGB intensities either manually through potentiometers or remotely through Bluetooth commands from a smartphone, while also providing an optional pseudo-random flickering effect for dynamic lighting ambience.

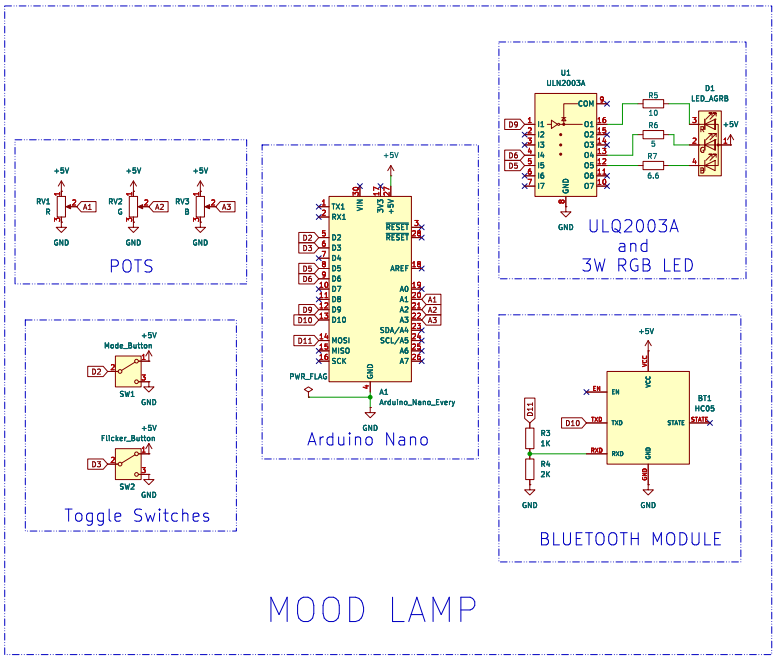
# 6 Components Used

| Reference | Value | Type | Power Rating | Tolerance |
|---|---|---|---|---|
| R1 | 1 kΩ | Carbon Film Resistor | 0.25 W | 5% |
| R2 | 2 kΩ | Carbon Film Resistor | 0.25 W | 5% |
| R3 | 10 Ω | Power Resistor | 1 W | 5% |
| R4 | 5 Ω | Power Resistor | 0.5 W | 5% |
| R5 | 6.6 Ω | Power Resistor | 0.75 W | 5% |
| C1 | 0.1 $\mu$F / 50 V | Ceramic Capacitor | – | 10% |
| C2 | 0.1 $\mu$F / 50 V | Ceramic Capacitor | – | 10% |
| C3 | 0.1 $\mu$F / 50 V | Ceramic Capacitor | – | 10% |
| VR1 | 50 kΩ | Panel Mount Potentiometer | – | – |
| VR2 | 50 kΩ | Panel Mount Potentiometer | – | – |
| VR3 | 50 kΩ | Panel Mount Potentiometer | – | – |
| LED1 | – | High Power SMD RGB LED | 3 W | – |
| IC1 | Arduino Nano | Microcontroller Board | – | – |
| IC2 | ULQ2003A | Darlington Driver IC | – | – |
| IC3 | HC-05 | Bluetooth Module | – | – |
| SW1 | SPDT | Toggle Switch | – | – |
| SW2 | SPDT | Toggle Switch | – | – |
| CN1 | USB Type-C | Power Receptacle | – | – |
| PCB1 | – | Zero PCB (Main Control Board) | – | – |
| PCB2 | – | Zero PCB (LED Mounting Board) | – | – |
| HS1 | – | Aluminium Heat Sink | – | – |
| TP1 | – | Thermal Paste | – | – |
| J1 | – | JST Female Connector | – | – |
| J2 | – | 90° Female Berg Strip Connector | – | – |
| W1 | – | Hook-up Wires | – | – |
| W2 | – | Multi-strand Insulated Wires | – | – |
| M1 | – | Heat Shrink Tubing | – | – |
| M2 | – | Electrical Insulation Tape | – | – |
| ENC1 | – | IP65 Plastic Enclosure | – | – |
| DIF1 | – | Table Tennis Ball (Light Diffuser) | – | – |
| ADH1 | – | Two-part Epoxy Adhesive | – | – |
| HW1 | – | Screws and Nuts | – | – |
| LBL1 | – | 3D Printed Labels / Decals | – | – |

## Block Diagram



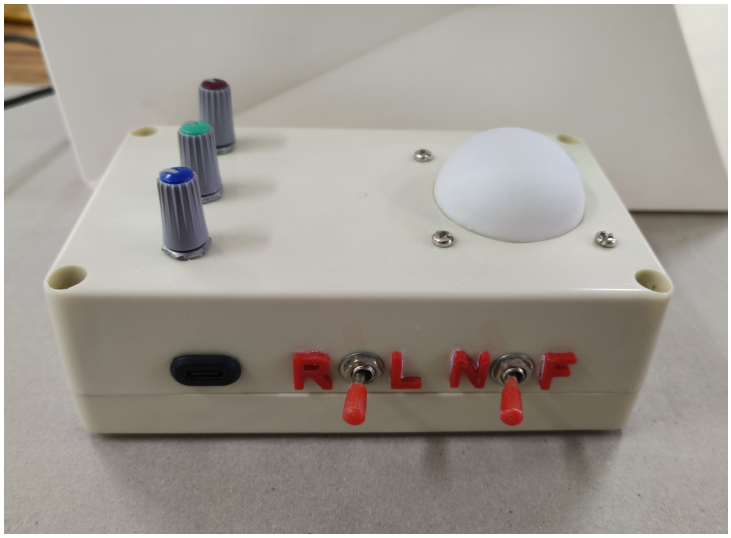

## Circuit Schematic



**Final Project Photograph 1**



**Final Project Photograph 2**



### Firmware

```
#include<SoftwareSerial.h>
#include <stdint.h> // For uint64_t

uint64_t lfsr;
uint8_t bit;

int modeButton = 2;
int flickerButton = 3;

int redPot = A1;
int greenPot = A2;
int bluePot = A3;
const uint8_t AVG_WINDOW = 8;
const uint8_t PWM_DEADZONE = 3;

int redPin=9;
int greenPin=6;
int bluePin=5;

SoftwareSerial BTSerial(10, 11); // RX, TX

uint8_t applyDeadzone(uint8_t PWM_Value) {
  return (PWM_Value < PWM_DEADZONE) ? 0 : PWM_Value;
}

void setup() {
  BTSerial.begin(9600);
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(modeButton, INPUT);
  pinMode(flickerButton, INPUT);

  // Generate a pseudo-random seed using analog noise
  lfsr = 0;
  for (int i = 0; i < 64; i++) {
    int noise = analogRead(A0);
    lfsr = (lfsr << 1) | (noise & 1);
    delay(20);
  }
  if (lfsr == 0) lfsr = 0xACE1uLL << 48 | 0x1234uLL; // Ensure it's not zero (LFSR
      gets stuck otherwise)

  BTSerial.println("Send RGB values:");
}

void loop() {

  // LFSR taps at 64, 63, 61, and 60 => bits 0, 1, 3, and 4
  bit = ((lfsr >> 0) ^ (lfsr >> 1) ^ (lfsr >> 3) ^ (lfsr >> 4)) & 1;
  lfsr = (lfsr >> 1) | ((uint64_t)bit << 63);

  if(digitalRead(modeButton)){
    // Rolling Average
    static uint8_t redArray[AVG_WINDOW];
    static uint8_t greenArray[AVG_WINDOW];
```

```
static uint8_t blueArray[AVG_WINDOW];
static uint16_t redSum = 0;
static uint16_t greenSum = 0;
static uint16_t blueSum = 0;
static uint8_t avgIndex = 0;
static bool avgFilled = false;

uint8_t redNew = analogRead(redPot) >> 2;
uint8_t greenNew = analogRead(greenPot) >> 2;
uint8_t blueNew = analogRead(bluePot) >> 2;

if (!avgFilled) {
  redArray[avgIndex] = redNew;
  greenArray[avgIndex] = greenNew;
  blueArray[avgIndex] = blueNew;
  redSum += redNew;
  greenSum += greenNew;
  blueSum += blueNew;
}
else {
  redSum -= redArray[avgIndex];
  greenSum -= greenArray[avgIndex];
  blueSum -= blueArray[avgIndex];
  redArray[avgIndex] = redNew;
  greenArray[avgIndex] = greenNew;
  blueArray[avgIndex] = blueNew;
  redSum += redNew;
  greenSum += greenNew;
  blueSum += blueNew;
}

avgIndex++;
if (avgIndex >= AVG_WINDOW) {
  avgIndex = 0;
  avgFilled = true;
}

uint8_t divisor = avgFilled ? AVG_WINDOW : avgIndex;

uint8_t avgRed = redSum/divisor;
uint8_t avgGreen = greenSum/divisor;
uint8_t avgBlue = blueSum/divisor;

if(digitalRead(flickerButton)){
  // Use LSB of LFSR to control High Power LED
  if(lfsr & 1){
    analogWrite(redPin, applyDeadzone(avgRed));
    analogWrite(greenPin, applyDeadzone(avgGreen));
    analogWrite(bluePin, applyDeadzone(avgBlue));
  }
  else{
    analogWrite(redPin, 0);
    analogWrite(greenPin, 0);
    analogWrite(bluePin, 0);
  }
  delay(20); // Delay between flickers (adjust to taste)
}
else{
```

```
      analogWrite(redPin, applyDeadzone(avgRed));
      analogWrite(greenPin, applyDeadzone(avgGreen));
      analogWrite(bluePin, applyDeadzone(avgBlue));
    }
  }
  else{
    static uint8_t rgb[3] = {0,0,0};

    if (BTSerial.available()) {
      for(int i=0; i<3; i++){
        rgb[i] = BTSerial.parseInt();
      }

      // now discard everything until newline (including CR/LF)
      while(BTSerial.find('\n')==0);

      // print values
      BTSerial.print("R: "); BTSerial.println(rgb[0]);
      BTSerial.print("G: "); BTSerial.println(rgb[1]);
      BTSerial.print("B: "); BTSerial.println(rgb[2]);
    }

    if(digitalRead(flickerButton)){
      // Use LSB of LFSR to control High Power LED
      if(lfsr & 1){
        analogWrite(redPin, rgb[0]);
        analogWrite(greenPin, rgb[1]);
        analogWrite(bluePin, rgb[2]);
      }
      else{
        analogWrite(redPin, 0);
        analogWrite(greenPin, 0);
        analogWrite(bluePin, 0);
      }
      delay(20); // Delay between flickers (adjust to taste)
    }
    else{
      analogWrite(redPin, rgb[0]);
      analogWrite(greenPin, rgb[1]);
      analogWrite(bluePin, rgb[2]);
    }

  }

}
```